CS6413 – Foundations of Privacy
Winter 2026

Design and Evaluation of Dynamic Searchable Symmetric Encryption (SSE) for Secure File Storage

Group 2

Muhammad Huzaifa Hameed – 3804782

Syed Aitezaz Imtiaz – 3783911

Hamza Rasool – 3776948

Muhammad Waleed – 3783369

Project 3 – Searchable Symmetric Encryption (SSE)
February 21, 2026

Abstract

This project focuses on designing and evaluating a **Searchable Symmetric Encryption (SSE)** system that allows users to store encrypted files while still being able to search them efficiently using keywords. The goal is to support dynamic updates such as adding, deleting, and modifying documents without rebuilding the entire index. So far, we have designed the system architecture and implemented a working prototype that supports encrypted storage, keyword-based search, and dynamic update operations. We have also developed an initial benchmarking framework and generated preliminary results for search latency and index size growth. The next phase of the project will focus on deeper performance comparison between static and dynamic SSE, extended evaluation with larger datasets, and final analysis of security trade-offs.

Project Goal and Scope

Problem Statement

Encrypting files in cloud or local storage protects confidentiality, but it prevents efficient keyword-based search. Users often need to retrieve specific documents without decrypting all stored data. The challenge is to design a Searchable Symmetric Encryption (SSE) system that allows secure and efficient search over encrypted files while supporting dynamic updates such as adding, deleting, and modifying documents.

Objectives

1. Design and implement a prototype SSE system for secure file storage.
2. Extend the system to support dynamic updates without rebuilding the entire index.
3. Evaluate performance metrics such as search latency, index size growth, and update overhead.
4. Compare static and dynamic SSE approaches based on efficiency and practicality.

Scope

The project includes implementing a working SSE prototype with encrypted storage, keyword-based search, dynamic updates, and performance benchmarking using synthetic datasets. It does not include full cloud deployment, multi-user access control, or advanced privacy features such as forward privacy or ORAM.

Success Criteria

The project will be successful if the system correctly supports encrypted search and dynamic updates, and if performance evaluation demonstrates reasonable scalability and efficiency across increasing dataset sizes.

Background and Related Work Progress

Searchable Symmetric Encryption (SSE) enables keyword search over encrypted data while preserving confidentiality. **Curtmola et al. (2006)** provided the formal security definitions and efficient inverted index construction that form the theoretical foundation of our project. Their work influenced our use of a keyword-to-document mapping and trapdoor-based search.

Kamara et al. (2012) introduced Dynamic SSE, allowing documents to be added and deleted without rebuilding the entire index. This directly guided our implementation of dynamic add, delete, and update operations.

Bost et al. (2016) proposed optimized SSE schemes for large datasets, emphasizing practical performance considerations. Their work motivated our evaluation of search latency and index size growth.

Based on these studies, we designed our system using an inverted index structure, cryptographic hashing for trapdoor generation, and dynamic update support. The literature also guided our decision to evaluate scalability and efficiency through performance benchmarking.

Methodology

We designed a prototype Searchable Symmetric Encryption (SSE) system consisting of three main components: encrypted file storage, a searchable inverted index, and a dynamic update mechanism. Files are encrypted using symmetric encryption before storage. Keywords are extracted from document content, hashed to generate secure search tokens (trapdoors), and mapped to document identifiers in the index.

To evaluate scalability, we use synthetically generated datasets with varying numbers of documents and keywords. The system is implemented in Python, using PyCryptodome for AES-GCM encryption and custom benchmarking scripts for performance testing.

From a privacy perspective, AES-GCM ensures confidentiality and integrity, while cryptographic hashing protects keyword information during search. Dynamic operations (add, delete, update) are supported to simulate realistic usage.

The evaluation plan includes measuring search latency, index size growth, and update overhead across increasing dataset sizes. These metrics will be used to compare static and dynamic SSE behavior and assess overall efficiency.

Work Completed So Far

Completed Tasks

- Designed overall system architecture for an SSE-based secure file storage system.
- Implemented encrypted file storage using AES-GCM.
- Developed inverted index structure for keyword-based search.
- Implemented trapdoor generation using cryptographic hashing.
- Added dynamic operations (add, delete, update) to support Dynamic SSE.

- Developed synthetic dataset generator for scalability testing.
- Implemented initial benchmarking framework for performance evaluation.

Current Outputs

- Working prototype supporting encrypted upload, search, delete, and update operations.
- System architecture diagram (high-level design).
- Initial benchmarking scripts for measuring search latency and index size growth.
- Preliminary performance graphs showing search latency and index scalability

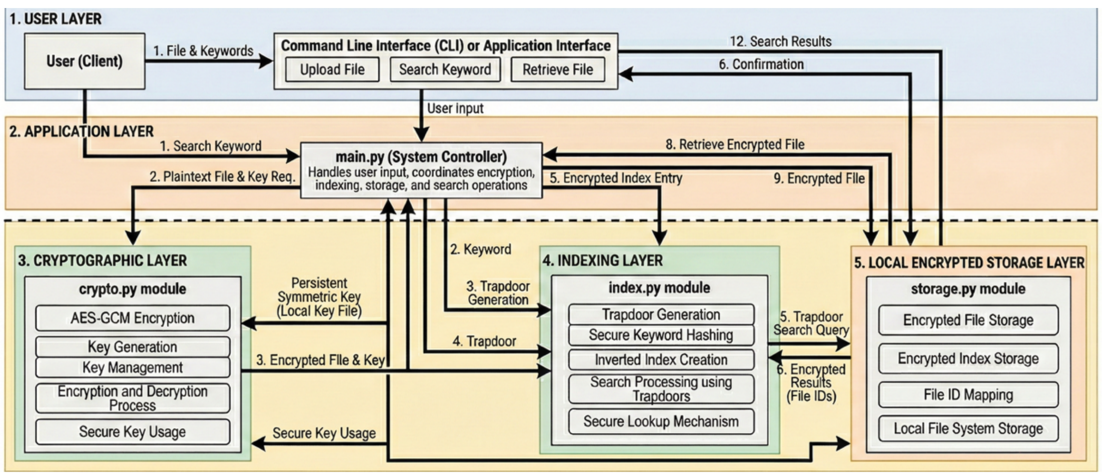

Figure 1*: High-level architecture of the proposed Dynamic SSE prototype. The current implementation follows this layered design, with performance evaluation and extended analysis ongoing.*

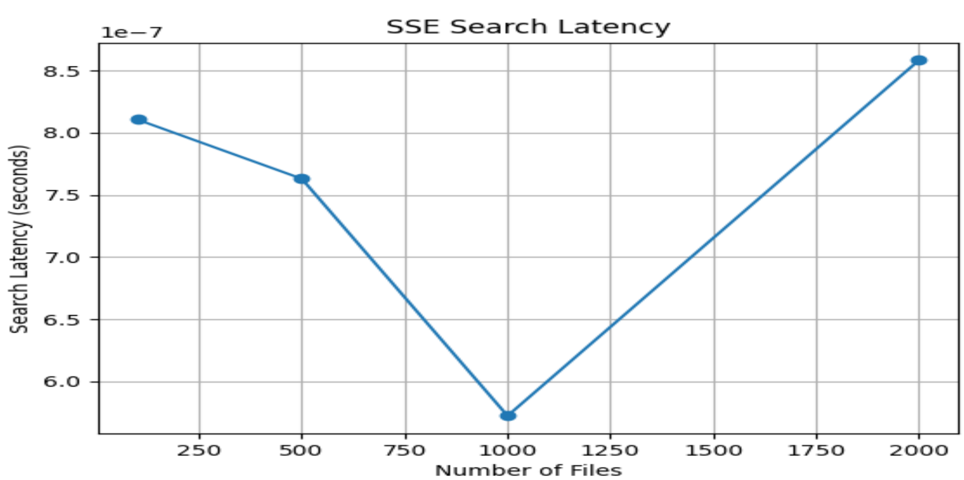

Figure 2: *Initial evaluation of search latency as the number of encrypted files increases.*

Current Status

Component	Status
System architecture design	Done
Encrypted storage implementation	Done
Search functionality	Done
Dynamic update operations	Done
Initial benchmarking framework	Done
Extended performance comparison	In progress
Security trade-off analysis	In progress
Final report preparation	Not started

Challenges, Risks, and Mitigation

Challenges

One technical challenge was maintaining index consistency during dynamic operations such as delete and update. Ensuring that keyword mappings were correctly modified without corrupting the index required careful handling of document–keyword relationships. Another challenge involved accurately measuring search latency at very small-time scales, as microsecond-level timing can be affected by system noise.

Risks

A potential risk is performance variability when testing with larger datasets, which may affect the stability of benchmark results. Another risk is limited scalability due to the use of JSON-based index storage in a local prototype environment.

Mitigation

To address timing variability, we use averaged measurements across multiple trials. To manage scalability concerns, dataset sizes are increased gradually and controlled through synthetic data generation. The modular design of the system also allows future improvements, such as replacing JSON storage with a database-backed index if needed.

Plan for the Remaining Weeks

Week	Focus	Key Tasks	Milestone

Week 1	Extended Evaluation	Larger dataset benchmarking Static vs dynamic comparison Validate latency, index growth, and update results	Baseline evaluation completed and comparative metrics finalized
Week 2	Analysis & Documentation	Analyze scalability and trade-offs Write Results & Discussion Add security/leakage discussion Finalize diagrams	Draft version of final report completed
Week 3	Finalization & Presentation	Review and edit report Verify benchmarks and figures Prepare slides and rehearse defense	Final report and presentation ready

Task Breakdown by Team Member

- **Syed Aitezaz Imtiaz:** Literature review refinement and security analysis (leakage, limitations, trade-offs).
- **Muhammad Huzaifa Hameed:** Benchmark validation, extended testing, and result verification.
- **Muhammad Waleed:** Report drafting (Methodology, Results, Discussion sections).
- **Hamza Rasool:** Diagram refinement, formatting, and presentation preparation.